Software Test Plan (STP) - Chat Application (Socket Programming)

Project: Chat Application (Socket Programming) - real-time WebSocket messaging

Version: 1.0

Authors: QA Team - Maansi S, Anand S, Darshan S Govekar, Vismaya G

Date: 02-10-2026

Status: Draft / For Review

Based on: SRS v1.0, SAD Component Design v1.0

1. Introduction

Purpose: This document defines the test plan for the Chat Application v1.0. It outlines the objectives, scope, strategy, resources, schedule and responsibilities for testing a Node.js and WebSocket based real-time messaging system.

Scope: Testing covers user registration/login/logout, session management, WebSocket connection establishment, presence, user search, one-to-one messaging with acknowledgement and timestamps, group chat, chat history, unread notifications, message validation, reconnection handling, standardized error handling, profile/status, and the non-functional and security requirements defined in the SRS (CHAT-F-001 to 016, CHAT-NF-001 to 007, PRJ-SR-001 to 006). Enterprise identity management, telecom services and production-scale moderation/billing are excluded, consistent with the SRS.

References: Chat Application SRS v1.0 (including RTM in Section 8); Chat Application SAD - Component Design v1.0; OWASP WebSocket and Application Security Testing guidance; RFC 6455 (The WebSocket Protocol); IEEE 829 test documentation structure.

Definitions: STP (Software Test Plan), SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), RTM (Requirements Traceability Matrix), WSS (WebSocket Secure, WebSocket over TLS), TLS (Transport Layer Security), AC (Acceptance Criteria), TC (Test Case), UAT (User Acceptance Testing), Room (logical group of users sharing messages), Presence (online/offline connection state), Ack (delivery acknowledgement).

2. Test Items

The test items are the six subsystems and 17 components defined in the SAD component diagram. Each component is unit tested in isolation and then integrated and tested through the subsystem interfaces.

Subsystem	Components (test items)	Related requirements
Client	ClientUI, ClientReconnectModule	CHAT-F-005, CHAT-F-014, CHAT-NF-003, CHAT-NF-005, CHAT-NF-007
Auth & Session	AuthModule, SessionManager, AuthorizationModule	CHAT-F-001, CHAT-F-002, PRJ-SR-002, PRJ-SR-003, PRJ-SR-004
Connectivity	WebSocketServer, TransportSecurity (WSS/TLS), ValidationModule	CHAT-F-003, CHAT-F-013, CHAT-NF-002, PRJ-SR-001, PRJ-SR-005
Messaging & Rooms	MessageRouter, RoomManager, PresenceModule, UserDirectory, NotificationModule	CHAT-F-004 to 010, CHAT-F-012, CHAT-NF-001

Subsystem	Components (test items)	Related requirements
Data & Profile	PersistenceModule / UserStore, ProfileModule	CHAT-F-011, CHAT-F-016
Error & Logging	ErrorHandler, LoggingModule	CHAT-F-015, PRJ-SR-006

3. Features to be Tested

Features are mapped to SRS requirement IDs. Priority is taken from the SRS.

Req ID	Feature / requirement	Priority
CHAT-F-001	User registration (unique username) and login authentication	High
CHAT-F-002	Logout and invalidation of session and WebSocket authorization	High
CHAT-F-003	WebSocket connection for authenticated clients; reject unauthorized or malformed attempts	High
CHAT-F-004	Online/offline presence updates on connect/disconnect	High
CHAT-F-005	User search / selection before a one-to-one chat	Medium
CHAT-F-006	Real-time one-to-one text messaging	High
CHAT-F-007	Delivery acknowledgement or error for every accepted message	High
CHAT-F-008	Group room creation and adding members	High
CHAT-F-009	Group message broadcast to connected, authorized members only	High
CHAT-F-010	Server-generated message timestamp	Medium
CHAT-F-011	Chat history storage and retrieval within retention limits	Medium
CHAT-F-012	Unread-message indicator for inactive conversations	Medium
CHAT-F-013	Rejection of empty, malformed or over-sized messages with clear error	High
CHAT-F-014	Client detection of disconnection and controlled reconnection	High
CHAT-F-015	Standardized error messages for all failure categories	Medium
CHAT-F-016	View and update profile display name / status message	Medium
CHAT-NF-001	Performance: 90% of accepted messages acknowledged within 1 s under normal load	High
CHAT-NF-002	Scalability: at least 100 concurrent connections without loss of accepted messages	High
CHAT-NF-003	Reliability: closed connection detected within 10 s; reconnection initiated	High
CHAT-NF-004	Availability: at least 99% of scheduled test duration	High
CHAT-NF-005	Usability: new user can register, log in and send a message without instructions	Medium
CHAT-NF-006	Maintainability: identifiable module separation (client, server, auth, routing, persistence)	Medium
CHAT-NF-007	Compatibility: current Chrome, Edge and Firefox	Medium
PRJ-SR-001	WSS/TLS enforced for non-local traffic	High
PRJ-SR-002	Authentication before protected WebSocket operations	High
PRJ-SR-003	Authorization of every message and room operation	High
PRJ-SR-004	No plaintext storage/logging of credentials; one-way password hashing	High

Req ID	Feature / requirement	Priority
PRJ-SR-005	Validation of message type, length, identifiers and room membership	High
PRJ-SR-006	Failure logging without passwords, session secrets or message content	High

4. Features Not to be Tested

- Internals of third-party libraries and runtimes (Node.js, the WebSocket library, the database engine, browser engines); they are assumed tested by their vendors.
- Enterprise identity management (SSO, LDAP), telecom services, and production-scale moderation or billing (out of scope per SRS Section 1.2).
- Native mobile applications; only responsive behaviour of the browser UI on common desktop and mobile browser sizes is checked as part of usability testing.
- Network infrastructure, ISP behaviour and production-grade TLS certificate issuance (a self-signed certificate is used in the test environment).
- Deployments beyond a single Node.js server and loads above 100 concurrent connections (outside SRS targets).

5. Test Approach / Strategy

The chat system is asynchronous and event-driven, so tests rely on programmatic WebSocket clients to send/receive events and assert on ordering, acknowledgements and timing, and on browser automation to verify the UI.

Levels

- **Unit tests (module-level):** Each SAD component is tested in isolation with mocks/stubs for its neighbours (e.g., MessageRouter with a stubbed PersistenceModule). Target: all public functions and error paths of every module.
- **Integration tests (subsystem interfaces):** Verify the interfaces shown in the SAD component diagram and the two sequence diagrams: Auth API, Session context, WebSocket API, validated events, and persistence/profile API (see Appendix B).
- **System tests (end-to-end):** Full flows through real browser clients and a running Node.js server: login to connect to chat to history to logout, group chat, reconnection.
- **Acceptance tests (UAT):** Executed against the SRS acceptance criteria and the Section 6 acceptance suites (functional, performance, security, usability) with the Project Guide / Course Coordinator.

Types

- **Functional testing:** All CHAT-F requirements including positive, negative and boundary cases (e.g., message of exactly the maximum length, empty message, unknown recipient).
- **Regression testing:** Automated smoke + core suite re-run on every new build and after each defect fix batch.
- **Performance and scalability testing:** Acknowledgement latency (CHAT-NF-001) and 100 concurrent connections (CHAT-NF-002) using a WebSocket load tool.
- **Reliability / availability testing:** Connection-drop and server-restart scenarios, plus a timed soak run for CHAT-NF-004.
- **Usability testing:** Task-based walkthrough with first-time users, clarity of feedback and error messages, responsive layout.

- **Compatibility testing:** Same system test subset on current Chrome, Edge and Firefox.
- **Security testing:** See Section 5.1.
- **Static inspection:** Code review and modularity inspection for CHAT-NF-006.

Entry and Exit Criteria

Entry Criteria: A stable build deployed to the test environment and passing the smoke test (server starts, a user can register, log in and connect); SRS and SAD baselined; test data and test accounts prepared; the configurable limits (maximum message length, room size, history retention) are documented.

Exit Criteria: 100% of planned test cases executed; all high-priority functional requirements implemented and verified; all PRJ-SR security requirements tested; 0 critical and 0 high defects open (medium/low defects have an agreed plan); RTM shows coverage for every requirement; CHAT-NF-001 to NF-004 measured results meet SRS thresholds; UAT sign-off obtained.

5.1 Security Validation

- **Transport (PRJ-SR-001):** Confirm non-local traffic is only accepted over wss://; capture traffic with Wireshark/DevTools to verify no plaintext frames; check TLS version and cipher suites (TLS 1.2+) using testssl.sh; verify plain ws:// is refused outside local development.
- **Authentication (PRJ-SR-002):** Attempt WebSocket connection and protected operations with no token, expired token, tampered token and a token reused after logout; all must be rejected with a standardized error.
- **Authorization (PRJ-SR-003):** Verify a user cannot send to, join, or read history of a room or conversation they are not a member of (horizontal privilege checks / ID tampering on conversation and room identifiers).
- **Credential protection (PRJ-SR-004):** Inspect the user store to confirm passwords are salted one-way hashes; search server logs and client messages for plaintext passwords and session tokens.
- **Input validation and fuzzing (PRJ-SR-005):** Fuzz message type, length, identifiers and JSON structure; send oversized frames, malformed JSON, binary frames, and script/HTML payloads (XSS) and confirm they are rejected or safely encoded; send message floods to confirm the server remains responsive (denial-of-service resilience).
- **Safe logging (PRJ-SR-006):** Trigger authentication and message-processing failures and inspect logs to confirm no passwords, session secrets or private message content are recorded.
- **Automated scanning:** OWASP ZAP (WebSocket and HTTP scan) against the running application, with findings triaged in the defect tracker.

6. Test Environment

Hardware: Server machine (Windows/Linux/macOS, minimum 4 GB RAM) capable of running Node.js and maintaining 100+ sockets; one or more client machines or the same machine for load generation; mobile device or browser device emulation for responsive checks.

Software: Chat Application v1.0 (Node.js server, WebSocket library, lightweight database or file store, browser client); latest stable Chrome, Edge and Firefox; self-signed TLS certificate for WSS testing.

Tools: Jest or Mocha (unit/integration), ws / wscat (scripted WebSocket clients), Selenium WebDriver (UI automation, cross-browser), Postman (optional REST endpoints), Artillery or k6 (WebSocket load/performance), OWASP ZAP and testssl.sh (security), Wireshark and browser

DevTools (traffic inspection, network throttling/offline mode), Git/GitHub, Jira or GitHub Issues (defect tracking).

Test Data: At least 120 dummy user accounts (including users with long names, special characters and duplicate-name attempts), pre-created rooms with varying membership, pre-seeded chat history across the retention boundary, and payload files for valid, empty, malformed and over-sized messages.

Environments: Local (ws:// on localhost) for unit/integration work; a LAN-based test environment with WSS enabled for system, security, performance and reliability testing.

7. Test Schedule

Proposed milestones; to be aligned with the course submission and evaluation dates.

Milestone	Planned date
Test plan review and approval	05-Oct-2026
Test case design (incl. RTM update)	06-Oct-2026 to 10-Oct-2026
Environment and test data setup	09-Oct-2026 to 12-Oct-2026
Smoke test / build acceptance	13-Oct-2026
Unit, integration and functional test execution	14-Oct-2026 to 20-Oct-2026
Performance, reliability and security test execution	21-Oct-2026 to 25-Oct-2026
Regression and cross-browser testing	26-Oct-2026 to 28-Oct-2026
User acceptance testing (UAT)	29-Oct-2026 to 30-Oct-2026
Test Summary Report	31-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual and automated) - catalogue in Appendix A
- Test Scripts (unit, WebSocket client, Selenium, load and security scripts)
- Test Data sets
- Updated Requirements Traceability Matrix (RTM status column: N/P/A)
- Test Execution Logs and performance result reports
- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Anand S	Prepare and maintain the test plan, coordinate execution, report status and metrics
Test Engineer(s)	Maansi S, Anand S, Darshan S Govekar, Vismaya G	Design and execute test cases, develop automation scripts, log defects
Developers	Maansi S, Anand S, Darshan S Govekar, Vismaya G (as assigned)	Fix defects, support triage, provide stable builds and configuration values

Role	Name	Responsibility
Course Coordinator	Prof. Rajeshwari Adarkatti	Final review and approval of the test plan and results

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early smoke builds from the development team; test completed modules first
Test environment downtime or machine unavailability	Keep a backup environment (second machine or cloud VM) and version-controlled setup scripts
Asynchronous, timing-dependent tests are flaky	Use explicit awaits on events, unique message IDs for correlation, and retry-with-logging before declaring a failure
Load generator and server share one machine, skewing CHAT-NF-001/002 results	Run load tool on a separate machine where possible; record hardware specs with each performance result
TLS/WSS configuration issues (self-signed certificate)	Prepare the certificate and browser trust settings early; verify during environment setup
SRS leaves some limits unspecified (max message length, room size, retention, retry intervals)	Obtain concrete configured values from developers before test case design and record them in the RTM comments
Browser behaviour differences across Chrome, Edge and Firefox	Run the compatibility subset early and on every release candidate
Team members act as both developers and testers (limited independence)	Cross-test: testers do not test their own modules; peer review of test cases

11. Assumptions & Dependencies

- The development team delivers testable builds on the agreed dates and fixes defects within the schedule.
- Configurable limits (maximum message size, room size, history retention, reconnect backoff) are defined before test case design.
- The persistence layer (database or file store) is available and can be reset to a known state between test runs.
- A browser-accessible client and a documented JSON message format and error catalogue (CHAT-F-015) are available.
- Acknowledgement latency (CHAT-NF-001) is measured from message send to server ack and, per the SAD, persistence is invoked asynchronously alongside delivery.
- Test users, rooms and certificates will be provided or created before execution starts.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment (server, database or network) is unavailable for more than 4 hours
- The build is too unstable (blocks more than 30% of test cases)
- A critical defect in authentication, session handling or WebSocket connection blocks all downstream testing
- The smoke test fails on a new build

Resume testing if:

- Blocking defects are fixed and verified on a new build
- The environment is stabilized and the smoke test passes

13. Test Case Management & Traceability

The RTM maps every SRS requirement to its test cases, so no requirement is left untested. Test case IDs follow the SRS convention (TC-<AREA>-nn); IDs marked "(added)" are additional negative/boundary cases introduced by QA and are listed in Appendix A. The RTM Status column in SRS Section 8 (N = Not tested, P = Planned/In progress, A = Achieved/Passed) will be updated during execution.

Req ID	Test case(s)	Test level / type
CHAT-F-001	TC-AUTH-01, TC-AUTH-03 (added)	Unit, System
CHAT-F-002	TC-AUTH-02	Unit, System
CHAT-F-003	TC-CONN-01, TC-CONN-02 (added)	Integration, System
CHAT-F-004	TC-PRES-01	Integration, System
CHAT-F-005	TC-SEARCH-01, TC-SEARCH-02 (added)	Unit, System
CHAT-F-006	TC-MSG-01	Integration, System
CHAT-F-007	TC-MSG-02	Integration, System
CHAT-F-008	TC-GRP-01	Unit, System
CHAT-F-009	TC-GRP-02	Integration, System
CHAT-F-010	TC-MSG-03	Unit, Integration
CHAT-F-011	TC-HIST-01, TC-HIST-02 (added)	Integration, System
CHAT-F-012	TC-NOTIF-01	Unit, System
CHAT-F-013	TC-VAL-01	Unit, Integration
CHAT-F-014	TC-REL-01	System
CHAT-F-015	TC-ERR-01	Unit, System
CHAT-F-016	TC-PROF-01	Unit, System
CHAT-NF-001	TC-PERF-01	Performance
CHAT-NF-002	TC-PERF-02	Performance
CHAT-NF-003	TC-REL-01	System
CHAT-NF-004	TC-REL-02	Reliability (soak)
CHAT-NF-005	TC-UX-01	Usability / UAT
CHAT-NF-006	INS-ARCH-01 (code review)	Static inspection
CHAT-NF-007	TC-COMP-01	Compatibility
PRJ-SR-001	TC-SEC-01	Security
PRJ-SR-002	TC-SEC-02	Security
PRJ-SR-003	TC-SEC-03	Security
PRJ-SR-004	TC-SEC-04	Security
PRJ-SR-005	TC-SEC-05	Security
PRJ-SR-006	TC-SEC-06	Security

14. Test Metrics & Reporting

Metrics collected:

- % of test cases executed
- % passed / failed / blocked
- Defect density (defects per module)
- Defect aging (open time by severity)
- Requirement coverage (from RTM)
- Acknowledgement latency (p50, p90, p99) and message loss count from performance runs
- Server availability % over the soak test window

Reports: Daily execution status; performance and security result reports; final Test Summary Report.

Defect Severity Definitions

Severity	Definition	Example
Critical	System unusable or security breach; no workaround	Any user can read another user's private messages; server crashes on login
High	Major feature fails; workaround difficult	Group messages not delivered; ack never returned
Medium	Feature partly affected; workaround exists	Unread indicator not cleared until refresh
Low	Cosmetic or minor issue	Misaligned timestamp, typo in error text

Appendix A: Test Case Catalogue

Summary of planned test cases. Detailed steps, test data and expected results will be elaborated in the test case design phase.

Test case ID	Req	Description	Expected result
TC-AUTH-01	F-001	Register a new user, then log in with valid credentials	Account created; authenticated session token issued
TC-AUTH-02	F-002	Log out, then reuse the old token and the open socket	Session invalidated; socket closed; old token rejected
TC-AUTH-03 (added)	F-001	Register duplicate username; log in with wrong password / unknown user	Standardized auth error; no session created
TC-CONN-01	F-003	Connect to WebSocket with a valid session token	Connection established; client shows connected state
TC-CONN-02 (added)	F-003	Connect with no, invalid or malformed token / handshake	Connection rejected with error
TC-PRES-01	F-004	User A connects/disconnects while User B has A in contact list	B sees A online, then offline, without refresh
TC-SEARCH-01	F-005	Search for registered user by partial name	Matching registered users returned and selectable
TC-SEARCH-02 (added)	F-005	Search for non-existent user / empty query	Empty result or clear message; no error
TC-MSG-01	F-006	A sends text message to B (both online)	B receives message in real time without refresh
TC-MSG-02	F-007	Send messages to online recipient, offline recipient and invalid recipient	Sender receives ack or error for every attempt
TC-MSG-03	F-010	Send message and inspect delivered/stored payload	Server-generated timestamp present; client-supplied time ignored
TC-GRP-01	F-008	Create room and add permitted members	Room created; members added; non-permitted add rejected
TC-GRP-02	F-009	Send group message with members online/offline and a non-member	Only connected authorized members receive it
TC-HIST-01	F-011	Retrieve history of a 1:1 chat and a room after relogin	Recent messages displayed in order
TC-HIST-02 (added)	F-011	Request history beyond retention limit and of an unauthorized conversation	Older messages not returned; unauthorized request denied
TC-NOTIF-01	F-012	Receive message while a different conversation is active	Unread indicator shown; cleared when opened
TC-VAL-01	F-013	Send empty, malformed JSON, and over-sized messages; boundary at max length	Invalid ones rejected with validation error; max length accepted
TC-REL-01	F-014, NF-003	Drop network / kill server while connected	Disconnect detected within 10 s; controlled retries begin and succeed on recovery
TC-ERR-01	F-015	Trigger each failure type (auth, bad format, unavailable recipient, unauthorized room, server error)	Each returns documented standardized error type

Test case ID	Req	Description	Expected result
TC-PROF-01	F-016	Update display name and status; relogin	Changes persist and display correctly to others
TC-PERF-01	NF-001	Normal-load run (Artillery/k6) measuring send-to-ack time	At least 90% of acks within 1 s
TC-PERF-02	NF-002	Ramp to 100 concurrent connections with active messaging	All connections stable; no accepted message lost (sent = acked = delivered)
TC-REL-02	NF-004	Soak run for scheduled duration (e.g., 60 min) with periodic health checks	Availability at least 99% (e.g., under 36 s downtime in 60 min)
TC-UX-01	NF-005	First-time users register, log in and send a message without help	Task completed unaided; feedback is clear; responsive layout works
INS-ARCH-01	NF-006	Code review of module boundaries against SAD	Client, server, auth, routing and persistence are separate modules
TC-COMP-01	NF-007	Run core system flows on Chrome, Edge and Firefox	Identical correct behaviour on all three
TC-SEC-01	SR-001	Inspect traffic and attempt ws:// on non-local interface; testssl.sh scan	Only WSS/TLS 1.2+ accepted for non-local traffic
TC-SEC-02	SR-002	Protected operations without, with expired and with tampered token	All denied
TC-SEC-03	SR-003	Send to / join / read a room or chat the user is not part of; tamper IDs	All denied; no data leaked
TC-SEC-04	SR-004	Inspect user store and logs after registration/login	Passwords stored as salted one-way hashes; none in plaintext or logs
TC-SEC-05	SR-005	Fuzz type/length/ID fields; XSS payloads; message flood	Rejected or safely encoded; server stays responsive
TC-SEC-06	SR-006	Trigger auth and message failures; review logs	Failures logged without passwords, session secrets or message content

Appendix B: Integration Test Scenarios (from SAD)

ID	Interface under test	Scenario	Expected result
INT-01	Auth & Session <-> Connectivity (Sequence 5.1)	Login, receive token, connect; WebSocketServer validates token via AuthModule; PresenceModule set online	Connection established and presence online; invalid token rejected
INT-02	Connectivity -> Messaging (Sequence 5.2)	sendMessage passes ValidationModule, then MessageRouter, then delivery to recipient	Valid message routed; invalid message never reaches the router
INT-03	Messaging -> Data & Profile	MessageRouter stores message with timestamp via PersistenceModule	Message persisted and retrievable; ack not blocked by persistence
INT-04	Auth & Session -> Messaging	AuthorizationModule checks membership for 1:1 and room operations	Unauthorized operation blocked before reaching RoomManager/MessageRouter

ID	Interface under test	Scenario	Expected result
INT-05	Connectivity -> Messaging (presence)	Socket open/close events update PresenceModule and notify relevant users	Presence updates delivered to relevant users
INT-06	All subsystems -> Error & Logging	Failures in each subsystem produce ErrorHandler responses and LoggingModule entries	Standard error returned; log entry free of secrets
INT-07	Client <-> Connectivity	ClientReconnectModule retries after connection loss and restores session	Reconnected with valid session; no duplicate messages